Security Vulnerability Remediation & Verification Report

Muttaka (Coworking Pass Platform)

Anas Saeed Alghamdi | Student ID: 444005852

Umm Al-Qura University
Prepared for the academic / field project supervisor
Report date: 8 November 2026 | Report reference: MTC-SEC-2026-11-08

1 Executive Summary

The four security findings raised during the supervisor audit have been addressed in the application code and supported by local automated verification. The remediation strengthens administrator authentication, rejects manipulated monetary inputs, makes checkout pricing and wallet settlement server-controlled, and adds persistent OTP abuse protection.

The recorded verification run passed 109 automated tests across 13 test suites. Backend and frontend TypeScript checks also passed. These results support technical closure at the implementation and local-verification stage. Production acceptance remains conditional on the Neon database migration and staging validation described in this report.

ID	Audit finding	Technical status
SEC-01	Weak admin security and default secrets	Remediated Local checks passed
SEC-02	Negative payment and decimal manipulation	Remediated SQL migration prepared
SEC-03	Payment and pricing security	Remediated Wallet-only settlement
SEC-04	OTP brute-force and flooding	Remediated Account cooldown and lockout

Scope and acceptance boundary

The evidence covers the Next.js App Router and TypeScript application, Prisma data access, PostgreSQL migration source, and the OTP frontend controls. Database transactions are mocked in the unit tests. Live PostgreSQL concurrency, migration compatibility, browser end-to-end behavior and a production build have not yet been verified.

Approval status: submitted for supervisor review. No approval or signature is implied by this report.

2 Detailed Technical Breakdown

SEC-01 Administrator Security and Secrets

IMPLEMENTATION REMEDIATED | LOCAL VERIFICATION PASSED

Problem statement

A missing or predictable JWT signing key could undermine session-token trust. Public seed credentials could also expose a privileged administrator account, particularly if development provisioning were enabled in a production deployment.

Root cause

Token signing and verification relied on environment values without fail-closed validation. Legacy seed paths contained a known administrator password, and Super Admin provisioning did not consistently enforce a sufficiently strong password composition policy.

Implemented solution

A shared secret validator now rejects undefined keys, keys shorter than 32 characters, insecure placeholders and obviously repetitive values. Both token signing and verification use the validated value; verification is restricted to HS256. Startup instrumentation loads the verifier outside the best-effort schema-sync exception handler, so invalid JWT configuration aborts startup.

Production rejects development administrator provisioning and the demo-account seed script. Loyalty-rule seeding no longer creates a fallback administrator. Provisioning revokes the legacy public password where applicable, and the migration revokes existing administrator sessions and pending OTPs.

Super Admin provisioning, login and password reset enforce a minimum of 12 characters containing uppercase letters, lowercase letters, numbers and symbols. A 72-byte ceiling avoids bcrypt password truncation. This is a length and composition requirement; it is not a measured guarantee of password entropy.

Verification tests

Test condition	Verified behavior
Missing, short or default JWT key	Validator rejects configuration with an error.
Weak administrator password	Provisioning policy rejects passwords missing required length or character classes.
Valid secret and administrator password	Positive controls are accepted.
Production development-seed selection	Administrator plan excludes the development default.

Implementation evidence

backend/lib/auth/secrets.ts; verify-token.ts; sessions.ts; admin-password.ts; super-admin.ts
backend/instrumentation.ts; backend/scripts/seed-accounts.mjs
Tests: security-input.test.ts and super-admin.test.ts

Closure assessment: the weak/default credential paths are closed in the implementation. Production deployment must supply a new random JWT key and approved administrator credentials before startup.

SEC-02 Negative Payment and Decimal Manipulation

IMPLEMENTATION REMEDIATED | DATABASE ACTIVATION PENDING

Problem statement

Negative, non-finite or otherwise malformed monetary values could produce invalid wallet movements or payment records. Uncontrolled numeric coercion and rounding could also conceal input formats that should be rejected.

Root cause

Financial routes applied inconsistent validation to client-supplied amounts. Parsing JSON before inspecting its numeric representation also removes the distinction between an ordinary number such as 100 and scientific notation such as 1e2. Database ledger tables lacked the additional check constraints delivered with this patch.

Implemented solution

financialBody() examines numeric JSON tokens before parsing. Quoted strings are handled separately, so descriptive text containing an exponent-like sequence is not incorrectly rejected. Numeric exponent notation, malformed JSON, non-object payloads and invalid amount types produce HTTP 400 responses.

The shared money() validator accepts only finite JSON numbers in the inclusive range of 1.00 to 100,000.00 SAR. Bounds are checked before rounding; accepted amounts are rounded to two decimal places. Numeric strings, null, NaN, infinities, zero and negative values are rejected.

The migration adds non-negative, finite-amount CHECK constraints to Payment, WalletTransaction and CompanyWalletTransaction. Historical zero-value records remain permissible at the database layer, while new monetary requests require at least SAR 1. Invalid existing ledger rows must be reconciled before migration; financial history is not silently rewritten.

Verification tests

Representative input	Verified result
-100, 0, 0.999, null and "100"	Rejected by monetary validation.
NaN, Infinity and 100000.01	Rejected as non-finite or out of range.
{"amount":1e2} and {"amount":1E+2}	Rejected before JSON representation is lost.
1.005 and 18.126	Rounded to 1.01 and 18.13 SAR.
1.00 and 100000.00	Accepted at the inclusive boundaries.

Implementation evidence and closure

backend/lib/security/input.ts
backend/prisma/migrations/20261108090000_security_hardening/migration.sql
Tests: security-input.test.ts and security-checkout.test.ts

Application validation is verified locally. The SQL constraints are prepared in migration source but have not been installed or tested on Neon. Direct database rejection tests are a mandatory deployment acceptance step.

SEC-03 Payment and Pricing Security

IMPLEMENTATION REMEDIATED | WALLET-ONLY CHECKOUT

Problem statement

Client-controlled prices and payment-success assertions could bypass legitimate payment. Retried operations could repeat a charge.

Root cause

Mock endpoints and separate booking/payment/wallet calls lacked a single authoritative settlement boundary and durable request-replay protection.

Implemented solution

Direct-booking prices now derive from stored section day, month or year rates, validated duration units and seats. Hourly prices use the section hourly rate and actual start/end interval, or a matching section package price. Subscription prices and terms come from MembershipPlan records. Client display text and submitted totals cannot replace the authoritative price.

Idempotency-Key is mandatory for the protected checkout POST routes. Keys are scoped to the authenticated caller and operation. PostgreSQL transaction advisory locks serialize matching requests. The request fingerprint and response are stored with the transaction: identical retries replay the saved result; reuse with a different payload returns HTTP 409.

Booking or subscription creation, wallet debit, successful WALLET payment and idempotency result commit in one Prisma transaction. Insufficient balance rolls back the booking. Company wallets use conditional balance decrements; personal wallets retain the existing company-credit spending controls.

Public payment registration can only return an existing settlement. Production mock-payment and simulated gateway-refund routes return HTTP 404. Development mock payments cannot credit real ledgers. Purchased subscription fields are protected from client updates, and refunds require an actual original payment.

Verification tests

Scenario	Verified result
Client sends a low price / total	Direct, hourly and subscription routes use database-derived prices.
Concurrent identical retry	Handler executes once; saved response is replayed.
Same key, different payload / caller	Conflicting payload returns 409; caller scope is isolated.
Insufficient wallet balance	Settlement throws before a successful payment is recorded.
Production mock-payment request	Endpoint returns HTTP 404.

Operational effect and closure

Wallet-funded checkout is the supported settlement path. Card/invoice payment, ordinary self-service wallet top-ups and browser-calculated pass/points discounts remain blocked until verified provider callbacks or server-side entitlement rules are implemented. This restriction is a deliberate mitigation, not a completed payment-provider integration.

Evidence: backend/lib/security/idempotency.ts and checkout.ts; direct-bookings, hourly-bookings, subscriptions and payments routes. Tests: security-pricing.test.ts, security-idempotency.test.ts and security-checkout.test.ts. Real PostgreSQL concurrency validation remains pending.

SEC-04 OTP Brute-Force and Flooding Protection

IMPLEMENTATION REMEDIATED | ACCOUNT-BOUND ENFORCEMENT

Problem statement

Repeated OTP requests could flood an account with messages, while automated guesses could target the six-digit verification code. Concurrent requests across application instances could bypass protections based on unsynchronized counters.

Root cause

The previous resend interval and verification flow did not provide the required persistent 60-second cooldown and 15-minute account lockout. Failure increments, code replacement and code consumption required a shared atomic control to remain consistent across workers.

Implemented solution

OTP issuance and verification now share a PostgreSQL transaction advisory lock per account. The OtpThrottle record stores the last issuance time, consecutive failures and lockout deadline. A new code cannot be issued within 60 seconds, including across different OTP purposes.

Five consecutive incorrect submissions invalidate pending codes and lock both verification and new issuance for 15 minutes. Requesting a new code does not clear the failure count. A correct consumed code can be used only once. Lockout responses use HTTP 429 with Retry-After and structured retryAfter/code fields.

The frontend OTP screen displays the resend countdown, consumes backend throttle responses, shows remaining lockout time and disables resend/verification while locked. The lockout deadline persists in session storage across reloads. Clearing browser state cannot bypass backend enforcement. Production also refuses to log OTP codes as a substitute for configured email delivery.

Verification tests

Scenario	Verified result
Concurrent requests for a fresh OTP	One issuance succeeds; the other observes cooldown.
Resend across OTP purposes	The same account cooldown remains in force.
Five concurrent wrong guesses	Codes are invalidated and a 900-second lockout is returned.
New code after four failures	The next wrong attempt still triggers lockout.
Correct code replay / lockout expiry	A consumed code works once; a new code works after expiry.

Implementation evidence and closure

backend/lib/auth/otp.ts and the authentication issue/verify routes
frontend/app/Auth/page.tsx; frontend/lib/api-config.ts
Tests: security-otp.test.ts; frontend and backend TypeScript checks

Account-level abuse protection is verified in local unit tests. A separate trusted-edge IP limiter is not included. Frontend timer behavior is implemented and type-checked; browser end-to-end validation and real database race testing remain deployment acceptance tasks.

3 Quality Assurance Summary

The recorded local verification run completed on 8 November 2026. All 109 automated tests passed across 13 test suites (test files), comprising 52 new security tests and 57 existing regression tests. The following results are the evidence for implementation-stage closure.

Quality gate	Result	Evidence / scope
Automated unit tests	PASS	109 passed; 13 test suites; backend npm test.
Backend TypeScript	PASS	npm run typecheck completed without type errors.
Frontend TypeScript	PASS	npm run typecheck completed without type errors.
Patch whitespace check	PASS	git diff --check completed successfully.
Prisma client generation	COMPLETE	Client regenerated for the schema additions.
Neon migration and live constraints	PENDING	Migration source prepared; no production application.
PostgreSQL concurrency / browser E2E	PENDING	Not established by mocked database unit tests.
Production build and deployment	PENDING	No production build or deployment executed.

Automated test inventory

Test suite	Passed	Test suite	Passed
security-input	36	security-otp	4
security-idempotency	4	security-checkout	4
security-pricing	4	super-admin	5
password-policy	19	sessions-credit	4
otp-mailer	6	capacity	5
fair-use	4	operating-hours	10
cr-number	4	TOTAL	109

Evidence interpretation

The unit tests use mocked transaction clients to verify validation, pricing, replay/conflict handling and OTP state transitions. They do not independently demonstrate PostgreSQL lock behavior, migration compatibility or runtime UI synchronization. Those boundaries are explicitly retained in the deployment acceptance plan.

Evidence record: SECURITY_PATCH.md; backend/tests/security-*.test.ts; the existing backend test suites; and the successful test/type-check command outputs recorded during remediation. This report does not certify unrelated platform routes or imply an independent production penetration test.

4 Deployment Recommendations

Database and configuration readiness

1. Reconcile negative or non-finite historical ledger values in staging. Verify catalog rates and set each MembershipPlan.durationDays to its actual term; the new field defaults to 30 days.

2. Apply the reviewed Prisma migration against an isolated staging database first, using the deployment environment for DATABASE_URL. Do not substitute prisma db push: it does not install the custom SQL CHECK constraints. Regenerate the client and repeat verification before production rollout.

Migration: backend/prisma/migrations/20261108090000_security_hardening/migration.sql

```
cd backend
npx prisma migrate deploy
npx prisma generate
```

3. Generate a new random JWT signing key, store it in the deployment secret manager as JWT_SECRET, and configure strong Super Admin credentials and OTP email delivery. Do not commit the generated key. Rotate any exposed/default key and revoke affected sessions.

```
node -e "console.log(require('crypto').randomBytes(48)
  .toString('base64url'))"
```

Run the generation command as one line. The report intentionally contains no usable secret.

Release acceptance

4. Verify real simultaneous retries, conflicting keys, insufficient-balance rollback, direct SQL constraint rejection, OTP cooldown/lockout, cancellation/refunds and the browser wallet flow. Run the production build. Deploy the matching database, backend and frontend only after these checks pass. Retain wallet-only restrictions until secure integrations are accepted.

5 Formal Sign-off and Approval

Student declaration: I submit the implemented remediation and recorded local verification evidence for review. I acknowledge the pending deployment acceptance steps and the checkout restrictions stated in this report.

Student	Supervisor
Anas Saeed Alghamdi Student ID: 444005852 Umm Al-Qura University	Name: _____ Title: _____ Department: _____
Signature: _____ Date: _____	Signature: _____ Date: _____

Supervisor disposition: [ ] Implementation closure accepted [ ] Revision required

Production release acceptance: [ ] Pending [ ] Accepted after deployment verification

Comments / conditions: _____
_____